

Universidad ORT Uruguay

Facultad de Ingeniería

Inteligencia Artificial

Documentación

Integrantes:

Juan Diego Fagnoni: 288818

Francisco Baraibar: 285465



Grupo: M7A

Docentes: Federico Vilensky, Martin Iturbide

2025

1) Módulo de Balance Dinámico (Cartpole-v1).....	3
1.1) Introducción.....	3
1.2) Entendimiento del problema: Cartpole-v1.....	3
Características del entorno.....	3
1.3) Exploración y análisis del entorno.....	3
Definición de rangos óptimos para discretización.....	4
1.4) Estrategias de Discretización Implementadas.....	4
1.5) Algoritmos Implementados.....	5
Q Learning clásico.....	5
Stochastic Q Learning.....	6
1.6) Exploración de Hiperparámetros.....	7
Hiperparametros extra para Stochastic Q Learning.....	7
Total de Experimentos planificados.....	8
1.7) Análisis de Resultados.....	9
Mejores configuraciones por algoritmo.....	9
Comparación entre Q-Learning y Stochastic Q-Learning.....	10
1.8) Análisis de Hiperparámetros importantes y sensibilidad.....	11
Parámetros específicos de Stochastic Q-Learning (k y memory).....	12
1.9) Conclusiones Módulo de Balanceo Dinámico.....	12
2) Módulo de Estrategia Cognitiva (MEC) – 2048.....	14
2.1) Introducción.....	14
2.2) Entendimiento del problema: 2048.....	14
2.3) Exploración y análisis del entorno.....	14
2.5) Modelado como búsqueda en árbol y agentes implementados.....	15
Agente Expectimax.....	15
Agente Minimax + poda Alpha-Beta.....	15
2.6) Plan de experimentación y benchmark.....	16
2.7) Resultados y análisis.....	16
Anexo.....	19
Anexo - Módulo de Balance Dinámico (Gráficas y Visualizaciones).....	19
Anexo - Módulo de Estrategia Cognitiva (Gráficas y Visualizaciones).....	24

1) Módulo de Balance Dinámico (Cartpole-v1)

1.1) Introducción

En esta sección de la documentación vamos a evidenciar y mostrar todo el proceso realizado para resolver el entorno Cartpole-v1 de Gymnasium.

El objetivo es poder detallar el proceso del trabajo de principio a fin, desde la comprensión del problema, el análisis del entorno y las decisiones tomadas, hasta los resultados obtenidos y las conclusiones finales.

Aclaraciones Importantes:

- En esta sección priorizamos la redacción del proceso. Las representaciones visuales fueron adjuntas en el anexo.
- El trabajo fue realizado principalmente sobre las dos discretizaciones iniciales, por eso la mayoría de comparaciones y visualizaciones del anexo no incluyen la tercer discretización. La discretización ultra gruesa fue implementada sobre el final, simplemente por el interés de verificar si era posible resolver el ejercicio aun con menos estados.

1.2) Entendimiento del problema: Cartpole-v1

Primero comenzamos investigando sobre el problema planteado. Este es un problema clásico de control donde se debe balancear un poste sobre un carro que se mueve horizontalmente. El agente puede aplicar fuerza hacia la izquierda o derecha y el objetivo es mantener el poste vertical el mayor tiempo posible (siendo el máximo 500 steps).

Características del entorno

El entorno presenta:

- 4 variables continuas que describen el estado del carro y son: posición del carro, velocidad del carro, ángulo del poste y velocidad angular.
- 2 acciones posibles: empujar el carro a la izquierda o a la derecha.
- Objetivo: alcanzar 500 steps sin que el poste se caiga

1.3) Exploración y análisis del entorno

En un comienzo analizamos el espacio de observaciones para entender qué variables teníamos y cuales eran más importantes para mantener el poste balanceado. Para esto corrimos 100 episodios aleatorios que nos permitieron ver que rango de valores reales tomaban dichas variables durante el juego.

Los resultados obtenidos fueron:

- Posición del carro: [-0.5473, 0.4346]

- Velocidad del carro: [-1.7464, 1.7417]
- Ángulo del poste: [-0.2094, 0.2093]
- Velocidad angular: [-2.3670, 2.6621]
-

Visualización de la distribución de las variables de observación

Luego de esto analizamos la criticidad de las variables, donde según la documentación de CartPole-v1, el episodio termina cuando:

- La posición del carro excede las +- 2.4 unidades
- El ángulo del poste excede +- 12 grados

Definición de rangos óptimos para discretización

Basándonos en el análisis realizado, los límites de terminación del entorno y buenas prácticas decidimos usar los siguientes rangos para hacer el trabajo:

Variables NO críticas

- Posición del carro: (-2.4, 2.4): estos valores corresponden al límite exacto de terminación del entorno
- Velocidad del carro: (-3.0, 3.0): en los episodios aleatorios obtuvimos +-1.7, decidimos ampliarlo para capturar mayores velocidades que pueda desarrollar un agente entrenado

Variables Críticas

- Ángulo del poste: (-0.21, 0.21): al ser la variable más crítica, usar el límite exacto asegura que todos los estados válidos este representados
- Velocidad angular del poste: (-3.5, 3.5): segunda variable crítica, el rango observado fue +-2,5, el cual ampliamos para capturar situaciones límite donde el agente debe realizar correcciones agresivas.

1.4) Estrategias de Discretización Implementadas

La letra solicitaba realizar la tarea con Q Learning. En CartPole-v1 las acciones ya son discretas (0 = izquierda, 1 = derecha), por lo tanto solo fue necesario discretizar el espacio de observaciones teniendo en cuenta la definición de rangos que habíamos hecho previamente. Para esto implementamos tres estrategias de discretización para comparar cómo afecta la granularidad al aprendizaje.

Inicialmente implementamos:

1) Discretización Gruesa:

Bins: asignamos 3 bins a las variables no críticas y 6 bins a las variables críticas
Total: 3x3x6x6: 324 estados

2) Discretización Fina

Bins: asignamos 6 bins a las variables no críticas y 12 bins a las variables críticas
Estados: 6x6x12x12: 5184 estados
Esta discretización captura más detalles pero entrena más lento y consume más memoria

[Visualización de comparación de bins \(gruesa vs fina\)](#)

Finalmente, luego de obtener los resultados de ambas discretizaciones tanto en Q Learning como en Stochastic Q Learning decidimos agregar una nueva discretización. Esto debido a que ambas discretizaciones dieron buenos resultados, entonces nos nació la duda ¿podemos resolver CartPole con menos estados? O aunque sea quedarnos cerca de los 500 steps. Por esto agregamos una discretización extra

3) Discretización Ultra Gruesa

Bins: asignamos 2 bins a las variables no críticas y 4 bins a las variables críticas
Estados: 2x2x4x4: 64 estados

1.5) Algoritmos Implementados

Los algoritmos implementados en el notebook fueron Q Learning y Stochastic Q Learning con el objetivo de compararlos y ver sus resultados.

En ambos enfoques partimos de la misma idea:

- Construimos una tabla Q de tamaño: Estados x Acciones
- Inicializamos la tabla Q en cero para todos los pares estado-acción

Al principio el agente “no sabe nada” y todos los valores $Q(s,a)$ representan la misma expectativa de recompensa

Teniendo esa base, cambiamos la forma de actualizar la tabla según el algoritmo.

Q Learning clásico

- 1) Primero observamos el estado continuo del entorno y lo transformamos en un estado discreto según la discretización correspondiente (primero utilizamos la gruesa, luego la fina y por último la ultra gruesa)
- 2) Elegimos una acción con una política ϵ -greedy:
 - Con probabilidad ϵ elegimos una acción al azar (exploración).
 - Con probabilidad $1-\epsilon$ elegimos la acción con mayor valor Q en ese estado (explotación).
- 3) Ejecutamos la acción en el entorno, recibimos una recompensa y el nuevo estado
- 4) Actualizamos la entrada $Q(s,a)$ con la fórmula:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

donde:

- α es la tasa de aprendizaje,
- γ es el factor de descuento,
- r es la recompensa inmediata,
- s' es el nuevo estado discreto,
- $\max_a Q(s',a)$ es el mejor valor posible en el nuevo estado

Para manejar la exploración usamos ϵ que decae en el tiempo:

- Empezamos con $\epsilon = 1.0$ (el agente explora casi siempre).
- Lo vamos reduciendo hasta un valor entre 0.01 y 0.1, donde casi todo el tiempo explota lo aprendido.-
- Probamos tanto decaimiento lineal como exponencial.

[Visualización de entrenamiento inicial de Q-Learning previo a ejecutar Grid searches](#)

En CartPole el cálculo de $\max_a Q(s,a)$ es barato porque solo hay 2 acciones, así que el coste por actualización es muy bajo.

Stochastic Q Learning

Tomando como referencia el paper, implementamos esta variante de Q-Learning, pensada para entornos con muchas acciones, donde evaluar todas en cada actualización resulta costoso.

Dicho esto la idea es aproximar el máximo usando solo un subconjunto aleatorio de acciones, donde la actualización pasa a ser:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot [r + \gamma \cdot \max_{a' \in A_k} Q(s',a') - Q(s,a)]$$

donde A_k es un subconjunto de tamaño k de las acciones disponibles.

En nuestra implementación:

- Mantenemos la estructura general: estados discretos, tabla Q inicializada en cero, política ϵ -greedy.
- La diferencia está en el cálculo del máximo, en lugar de mirar todas las acciones, miramos al subconjunto A_k .

Hiperparametros adicionales:

Además de α , γ y ϵ , Stochastic Q-Learning introduce:

- k (subset size): cuántas acciones consideramos al calcular el máximo.

Probamos $k = 1$ y $k = 2$.

- Con $k = 1$ sólo miramos una acción al azar del estado siguiente: es más rápido pero puede ser una mala aproximación del máximo.
- Con $k = 2$, en CartPole terminamos mirando las 2 acciones, o sea que el algoritmo se vuelve prácticamente equivalente al Q-Learning clásico.
- use_memory: si es True, el subconjunto A_k no es completamente aleatorio:
 - Siempre incluimos la última acción que funcionó bien (por ejemplo, la acción que se usó en el mejor valor Q reciente).
 - El resto de las acciones del subconjunto se eligen al azar.
 - Esto intenta reducir la varianza: no dependemos solo de acciones elegidas al azar, sino que arrastramos algo de “experiencia pasada”.

1.6) Exploración de Hiperparámetros

Para encontrar la configuración de hiperparámetros que obtenga mejores resultados, optamos por implementar un grid search sistemático sobre un conjunto de valores definidos. Esta estrategia nos permitió explorar de forma exhaustiva las combinaciones posibles, comparar el desempeño de cada configuración bajo la misma métrica de evaluación y fundamentar la elección final en evidencia empírica, y no en decisiones ad hoc.

Para ambos algoritmos usamos el mismo espacio base, con la intención de obtener una comparación “justa”:

Hiperparametros comunes

- Alpha (α) $\in [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]$, controla cuánto corregimos $Q(s,a)$ con cada nueva experiencia.
- Gamma (γ) $\in [0.95, 0.99]$, cuánto peso le damos al futuro frente a la recompensa inmediata.
- Epsilon final (ϵ_{end}) $\in [0.01, 0.05, 0.1]$, determina el nivel de exploración mínima al final del entrenamiento.
- Estrategia de decaimiento de $\epsilon \in \{\text{linear, exponential}\}$ cómo pasamos de $\epsilon=1.0$ al valor final (de forma uniforme o más agresiva al principio).

Combinando los valores de los hiperparametros “base” obtuvimos un total de 72 configuraciones para cada discretización en Q Learning

[Visualización del impacto de \$\alpha\$ y \$\gamma\$ \(heatmaps en Q-Learning\)](#)

Hiperparametros extra para Stochastic Q Learning

Como mencionamos anteriormente para esta estrategia agregamos:

k (subset_size) $\in [1, 2]$, es el número de acciones que miramos al aproximar el máximo:

- k=1: muy barato, pero puede aproximar mal el mejor valor.
- k=2: en CartPole equivale a mirar las dos acciones posibles, igual que en Q Learning clásico.

use_memory $\in [\text{True}, \text{False}]$, decidimos si incluir o no la última acción “buena” dentro del subconjunto de acciones, para aprovechar información histórica.

Sumando estos parámetros, terminamos teniendo 288 configuraciones para cada una de las discretizaciones en Stochastic Q Learning.

Total de Experimentos planificados

Agrupando por algoritmo y discretización, el plan de experimentos nos quedó:

Algoritmo	Discretización	Configuraciones	Tiempo total(Gridsearch)
Q-Learning	Ultra Gruesa	72	16 min
Q-Learning	Gruesa	72	27 min
Q-Learning	Fina	72	82 min
Stochastic Q-Learning	Ultra Gruesa	288	98min
Stochastic Q-Learning	Gruesa	288	183 min
Stochastic Q-Learning	Fina	288	402 min

TOTAL : 1080 configuraciones ejecutadas

Estos 6 grid Search nos permitieron evaluar un total de 1080 experimentos con distintos hiperparametros y sobre 2 tipos distintos de Q-Learning para encontrar la mejor solución para CartPole-v1.

Episodios de entrenamiento por discretización

Como cada discretizacion genera un espacio de estados distintos, ajustamos la cantidad de episodios de entrenamiento para cada una de ellas. Buscando una cantidad de episodios suficiente para entrenar bien, pero sin excedernos para no realizar episodios innecesarios

Discretización	# Estados	Episodios de entrenamiento	
Ultra Gruesa	64	3,500	Pocos estados, se recorre más rápido el espacio y converge antes.
Gruesa	324	7,500	Tamaño intermedio, se necesita más exploración para cubrir bien los estados.

Fina	5,184	12,000	Muchos estados, se requiere más episodios para visitar suficientes combinaciones.
------	-------	--------	---

Después de entrenar cada configuración, evaluamos el desempeño con una política totalmente greedy (sin exploración), donde ejecutamos 200 episodios de evaluación por modelo. En cada episodio la acción se selecciona siempre como $\text{argmax}_a Q(s,a)$. A partir de esos episodios calculamos recompensa promedio, desvío estándar y el porcentaje de episodios que alcanzaron el criterio de resolución del entorno. Haciendo esto logramos obtener un top de las mejores y de las peores configuraciones en cada uno de los grid search realizados. Luego de hacer una ejecución completa del notebook obtuvimos los resultados, los cuales veremos en la siguiente sección

1.7) Análisis de Resultados

En esta sección resumimos las mejores configuraciones encontradas para cada combinación de algoritmo y discretización, y analizamos cómo se comportaron Q-Learning clásico y Stochastic Q-Learning en términos de rendimiento, estabilidad y costo de entrenamiento.

Mejores configuraciones por algoritmo

Q-Learning Clásico

A partir del grid search, las mejores configuraciones para Q-Learning clásico fueron:

Discretización	Estados	α	γ	ϵ_{end}	Decay	Eval Mean $\pm$ Std	Success Rate	Tiempo
Ultra Gruesa	64	0.10	0.95	0.10	linear	493.83 $\pm$ 49.25	98.0%	8.3 s
Gruesa	324	0.10	0.95	0.05	exponential	498.38 $\pm$ 13.63	96.0%	48.7 s
Fina	5,184	0.10	0.99	0.05	linear	500.00 $\pm$ 0.00	100.0%	66.5 s

Como observamos en la tabla, en Q-Learning solo obtenemos solución perfecta con la discretización fina. De todas formas los resultados de la discretización Ultra Gruesa y Gruesa se acercaron bastante al objetivo, lo que nos permite darnos cuenta que estamos en un entorno donde no se necesita una gran cantidad de estados para alcanzar el éxito

[Visualización de comparación de resultados entre discretización gruesa y fina \(Q-Learning\)](#)

Stochastic Q-Learning

Por otro lado, las mejores configuraciones para Stochastic Q-Learning fueron:

Discretización	Estados	α	γ	ϵ_{end}	Decay	k	Mem	Eval Mean $\pm$ Std	Success Rate	Tiempo
Ultra Gruesa	64	0.10	0.99	0.05	linear	1	Sí	500.00 $\pm$ 0.00	100.0%	27.4 s
Gruesa	324	0.20	0.99	0.01	exp	2	No	497.74 $\pm$ 16.97	96.0%	92.4 s
Fina	5,184	0.10	0.99	0.01	exp	2	Sí	500.00 $\pm$ 0.00	100.0%	375.1 s

Para el caso de Stochastic Q Learning, alcanzamos resultados perfectos tanto con la discretización fina como con la ultra gruesa. La diferencia más destacable son los tiempos de ejecución, en la discretización ultra gruesa alcanzamos una solución perfecta en sólo 27 segundos.

Comparación entre Q-Learning y Stochastic Q-Learning

Discretización ultra gruesa (64 estados)

- Q-Learning clásico: 493.83 $\pm$ 49.25, 98.0% de éxito.
- Stochastic Q-Learning: 500.00 $\pm$ 0.00, 100.0% de éxito.

Aquí Stochastic Q-Learning es claramente superior: mismo espacio de estados, pero mayor estabilidad y éxito perfecto.

Discretización gruesa (324 estados)

- Q-Learning: 498.38 $\pm$ 13.63, 96.0% de éxito.
- Stochastic: 497.74 $\pm$ 16.97, 96.0% de éxito.

Ambos algoritmos logran rendimientos muy similares; Q-Learning tiene apenas menos varianza, pero no llega a ser una diferencia significativa.

Discretización fina (5,184 estados)

- Ambos alcanzan 500.00 $\pm$ 0.00, 100% de éxito.

En este escenario, el algoritmo ya no es el factor limitante. La discretización fina permite a los dos llegar a la solución perfecta.

[Visualización de comparación entre Q-Learning vs Stochastic Q-Learning \(gruesa vs fina\)](#)

1.8) Análisis de Hiperparámetros importantes y sensibilidad

Una vez observadas las mejores configuraciones, nos pareció importante detenernos en el impacto que tuvieron los distintos valores de los hiperparámetros en dichos resultados. Para eso analizamos las mejores 15 configuraciones de cada grid search para ver qué tan sensibles eran los resultados a los hiperparámetros y qué patrones se repetían.

Alpha (α)

En general, las configuraciones con mejor rendimiento se concentran en valores de α relativamente bajos:

- En Q-Learning clásico, las discretizaciones Ultra Gruesa y Gruesa se apoyan fuertemente en $\alpha = 0.10$, mientras que en la discretización Fina aparecen con frecuencia valores cercanos como $\alpha = 0.30$, junto a 0.10 y 0.20 .
- En Stochastic Q-Learning, los mejores modelos usan mayoritariamente α entre 0.10 y 0.20 , con alguna presencia de 0.30 y valores más altos mucho menos frecuentes.

Esto sugiere que CartPole se beneficia de un aprendizaje moderado, donde la tabla Q se ajusta de forma gradual y no con pasos muy agresivos.

Gamma (γ)

El factor de descuento mostró una influencia clara:

- En los tres grid search de Q-Learning, el valor más frecuente fue $\gamma = 0.99$, aunque $\gamma = 0.95$ también produce algunas configuraciones buenas.
- En Stochastic Q-Learning, observamos un patrón mixto:
 - Con discretización Ultra Gruesa, las mejores configuraciones se inclinan por $\gamma = 0.95$ (aunque la mejor fue con $\gamma = 0.99$).
 - Con discretizaciones Gruesa y Fina, predomina $\gamma = 0.99$.

En resumen, el rendimiento es bastante sensible a γ . En espacios de estados más grandes conviene usar un γ alto (0.99) para valorar fuertemente la recompensa futura, mientras que en espacios muy compactos (Ultra Gruesa) un $\gamma = 0.95$ es suficiente y obtiene resultados competitivos.

Epsilon final (ϵ_{end}) y tipo de decaimiento

A diferencia de α y γ , no apareció un único valor de ϵ_{end} “ganador” en todos los escenarios

En algunos grids (por ejemplo, Q-Learning Ultra Grueso) las mejores configuraciones se inclinan por $\epsilon_{\text{end}} = 0.01$, mientras que en otros (Q-Learning Gruesa) aparece con más frecuencia $\epsilon_{\text{end}} = 0.10$, y en varios casos $\epsilon_{\text{end}} = 0.05$ es una opción intermedia sólida.

Algo similar ocurre con el tipo de decaimiento (lineal vs exponencial): en ciertos grids predomina el lineal y en otros el exponencial, pero ambos aparecen dentro de las mejores configuraciones.

Esto indica que el sistema es menos sensible a ϵ_{end} y al tipo de decaimiento, siempre que:

- ϵ_{end} se mantenga en un rango razonable [0.01, 0.10], y
- ϵ vaya decayendo progresivamente desde 1.0 durante el entrenamiento.

Parámetros específicos de Stochastic Q-Learning (k y memory)

En Stochastic Q-Learning analizamos además el tamaño del subconjunto de acciones k y el uso de memoria (use_memory):

El tamaño del subconjunto $k = 2$ aparece como el valor más frecuente en los tres grids (Ultra, Gruesa y Fina), aunque $k = 1$ también logró incluirse en el top de configuraciones en varios casos, especialmente en discretizaciones gruesas o finas.

La opción use_memory = True aparece en una proporción importante de las mejores configuraciones, en particular en la discretización Ultra Gruesa y Fina, mientras que en algunos grids (como Gruesa) convive con variantes sin memoria.

En términos de sensibilidad, Stochastic Q-Learning resulta más delicado respecto a la elección de k y memory que a ϵ_{end} o al tipo de decaimiento:

- Usar $k = 2$ tiende a producir políticas más robustas que $k = 1$
- Activar memory suele ayudar a estabilizar el aprendizaje, aunque no es estrictamente imprescindible en todos los escenarios.

1.9) Conclusiones Módulo de Balanceo Dinámico

A modo de cierre de este ejercicio logramos obtener las siguientes conclusiones.

Si bien la discretización fina garantiza una solución perfecta con ambos algoritmos, Cartpole-v1 es un entorno relativamente sencillo que puede resolverse con menos estados, minimizando tiempos de ejecución.

Dicho esto, consideramos que la combinación más interesante desde el punto de vista práctico fue Stochastic Q-Learning con discretización ultra gruesa, que logra resolver el entorno con la tabla Q más pequeña y en tiempos de entrenamiento muy reducidos, manteniendo éxito perfecto.

En cuanto a los hiperparámetros, el análisis que hicimos nos indica que CartPole se entrena de forma estable con tasas de aprendizaje moderadas y factores de descuento altos mientras que ϵ_{end} y el tipo de decaimiento resultaron menos críticos dentro de rangos razonables. En Stochastic Q-Learning, usar $k = 2$ y activar memory suele conducir a políticas más robustas, esto debido a que la verdadera ventaja de Stochastic se encuentra en la aplicación del mismo en problemas con muchas acciones.

Una vez finalizado el proceso, guardamos los mejores 6 modelos (correspondientes a la mejor configuración para cada grid search) en formato .pkl en la carpeta models y respaldado en la carpeta models_backup para asegurar su entrega correcta.

Aclaración: nos referimos a mejor modelo al que dio mejor resultado en cada uno de los 6 grid search

2) Módulo de Estrategia Cognitiva (MEC) – 2048

2.1) Introducción

En este informe vamos a mostrar el proceso realizado para resolver el juego 2048 con agentes de búsqueda en árbol con funciones heurísticas de evaluación. Para esto implementamos las técnicas Expectimax y Minimax con poda Alpha-Beta, con funciones heurísticas que se combinan con pesos para evaluar el entorno del agente.

2.2) Entendimiento del problema: 2048

2048 es un juego por turnos sobre una grilla 4x4 donde en cada turno el agente elige una de cuatro acciones (arriba, abajo, izquierda, derecha). Si el movimiento es válido, las fichas se deslizan y las fichas iguales se fusionan, creando una ficha con valor doble. Luego el entorno agrega una ficha nueva de valor 2 o 4 con probabilidades 0.9 y 0.1 respectivamente, en una posición vacía de forma aleatoria. Esto implica que el agente deba tomar decisiones bajo incertidumbre para resolver el problema, por eso nuestro árbol de búsqueda combina nodos de decisión del agente con nodos probabilísticos.

2.3) Exploración y análisis del entorno

Luego de comprender las reglas del juego al que nos enfrentamos, analizamos implementar una solución que evite la búsqueda hasta estados terminales debido a que el árbol crece muy rápido por lo variable que es la fase estocástica del problema. Para esto implementamos un **corte por profundidad** en nuestra búsqueda, limitándola a una “depth” fija.

Además para este problema implementamos una función **evaluate(board)** que mide “qué tan bueno” es un estado intermedio.

2.4) Función de evaluación (heurísticas) y presets de pesos

Implementamos una función basada en una combinación lineal de criterios conocidos en 2048 que buscan capturar que tan bueno o prometedor es el tablero, definimos 6 heurísticas simples:

- **Espacios vacíos (count_empty)** : premiar un tablero con más celdas libres
- **Monotonidad (monotonicity)**: premiar grillas “ordenadas” (por filas/columnas), que facilitan construir fichas grandes sin romper estructura.
- **Ficha máxima en esquina (max_tile_in_corner)**: incentiva una estrategia estable (concentrar la ficha grande).
- **Smoothness (smoothness)**: penaliza diferencias grandes entre adyacentes (en escala log2) porque dificultan merges.
- **Merges posibles (merges_possible)**: premia oportunidades inmediatas de combinación.
- **Peso posicional snake (positional_weight)**: matriz de pesos decrecientes que favorece concentrar valores en una trayectoria ordenada.

También definimos distintos presets de pesos para la función de evaluación con el objetivo de comparar cómo varia el desempeño al priorizar diferentes intuiciones del juego:

- **baseline:** {"empty": 250, "monotonicity": 1.5, "corner": 25, "smoothness": 3.0, "merges": 50}
- **tuned:** {"empty": 320, "monotonicity": 2.0, "corner": 35, "smoothness": 2.0, "merges": 70}
- **corner_heavy:** {"empty": 300, "monotonicity": 1.8, "corner": 60, "smoothness": 2.0, "merges": 60}
- **smooth_heavy:** {"empty": 280, "monotonicity": 2.2, "corner": 25, "smoothness": 4.0, "merges": 50}
- **snake:** {"empty": 320, "monotonicity": 2.0, "corner": 40, "smoothness": 2.5, "merges": 70, "positional": 0.5}

2.5) Modelado como búsqueda en árbol y agentes implementados

Modelamos estados como la grilla 4x4 y construimos un árbol por capas dónde nuestros nodos MAX representan el turno del agente y elige la acción con mayor valor, nodos CHANCE (Expectimax) que representan el turno estocástico, aquí se calcula el valor esperado ponderando por probabilidades y finalmente nodos MIN (Minimax) que representan el turno de forma adversarial, se elige la inserción que minimiza el valor.

Aclaración en nuestra implementación:

Para no contaminar el azar del juego durante el “thinking” de los agentes, la interfaz Agent expone los métodos `safe_clone` y `safe_get_available_moves` para preservar el estado de `np.random`

Agente Expectimax

En Expectimax, los nodos donde el agente elige el movimiento son nodos MAX: itera movimientos válidos, simula con `clone.move(move)` y se queda con el mejor valor. Luego, el paso aleatorio de aparición de ficha se modela como un nodo CHANCE: en `_chance_value` enumera todas las celdas libres y pondera `tile=2` con 0.9 y `tile=4` con 0.1, usando probabilidad uniforme por celda.

En la práctica, nuestra función recursiva `_expectimax(board, depth, is_chance)` alterna entre estos nodos probando cada movimiento disponible y elige el que maximiza el subárbol a profundidad “depth” fija.

Agente Minimax + poda Alpha-Beta

En Minimax planteamos el problema como si el azar fuera un oponente que elige la peor inserción posible. Esto no refleja exactamente el 2048 real, pero nos permite comparar el enfoque “optimista” (Expectimax) contra uno “pesimista” (Minimax). Implementamos minimax con dos funciones:

- 1) `_max_value(board, depth, alpha, beta)` para el turno del agente.

- 2) **_min_value(board, depth, alpha, beta)** para el turno “adversarial” de inserción. En este caso enumeramos inserciones posibles (celda libre × {2,4}) y elegimos la que minimiza el valor.

Para la poda Alpha-Beta usamos la flag “use_pruning” y hacemos en los nodos MIN se poda si $\text{value} \leq \alpha$, y en los nodos MAX se poda si $\text{value} \geq \beta$.

En `play()`, inicializamos $\alpha = -\infty$ y $\beta = \infty$ y evaluamos cada movimiento con `_min_value(...)` para elegir el mejor. Además, dejamos la posibilidad de desactivar la poda (`use_pruning=False`) para comparar costo computacional con el mismo algoritmo base.

2.6) Plan de experimentación y benchmark

Para ejecutar comparaciones de manera sistemática implementamos un benchmark [bench.py](#) que permite automatizar corridas, y parametrizar y exportar resultados. Esto gracias a las variables experimentales que controlamos en cada corrida:

- Agente: random, expectimax o minimax
- Profundidad (depth): probamos profundidades 2 y 3 con varios episodios.
- Presets de pesos / pesos heurísticos: probamos los agentes con varios presets para comparar
- Poda Alpha-Beta (en Minimax): comparamos corridas con poda y sin poda para analizar impacto
- Seed: repetimos con distintas semillas las corridas para comparabilidad.
Implementamos esto para que el patrón de spawn de fichas sea reproducible y mejorar el valor de la comparación.

Guardamos métricas como win (si se alcanza 2048), max_tile alcanzada, moves totales, duration_sec y grid_sum que nos permiten analizar tanto “calidad de juego” como “costo”.

Protocolo de comparación

Corrimos el agente RandomAgent para tener los resultados como referencia para comparación, luego comparamos el impacto de variar la profundidad variando entre 2 y 3 para Expectimax (Minimax solo depth 3), comparamos algunos de los presets de pesos y en Minimax, medimos el impacto de la poda.

2.7) Resultados y análisis

A continuación resumimos los resultados agregados por configuración. Reportamos promedio ± desvío estándar (sobre episodios) y el porcentaje de wins.

Técnica	Preset	Setup	Depth	Poda (On/Off)	Episodios	Win%	Max tile (avg)	Moves (avg)	Tiempo (s)
expectimax	smooth_heavy	e10	2	-	30	16.7	998.4	748.9	37.2

expectimax	snake	e10	3	-	30	16.7	968.5	810.8	112.9
minimax	snake	prune_on_e10	3	On	30	3.3	759.5	658.4	17.8
minimax	snake	prune_off_e10	3	Off	10	0.0	691.2	647.3	45.2
expectimax	baseline	e10	3	-	30	0.0	622.9	634.4	126.6
expectimax	corner_heavy	e10	2	-	30	0.0	605.9	549.4	27.1
expectimax	corner_heavy	e10	3	-	30	0.0	588.8	615.4	124.2
expectimax	tuned	e10	3	-	30	0.0	554.7	594.0	121.5
minimax	smooth_heavy	prune_on_e10	3	On	30	0.0	546.1	559.8	21.5
expectimax	smooth_heavy	e10	3	-	30	0.0	537.6	586.8	122.3
expectimax	tuned	e10	2	-	30	0.0	524.8	510.4	26.1
minimax	smooth_heavy	prune_off_e10	3	Off	10	0.0	512.0	509.0	42.2
expectimax	snake	e10	2	-	30	0.0	371.2	397.9	11.8
random	-	e10	-	-	10	0.0	112.0	119.5	0.2

Comparación global de técnicas (calidad vs costo)

Mirando los promedios, Expectimax fue el que mejor jugó en general. Las mejores corridas fueron smooth_heavy con depth 2 (Win% 16.7 y max tile promedio 998.4) y snake con depth 3 (Win% 16.7 y max tile promedio 968.5).

En cambio, Minimax queda bastante más atrás en tasa de victoria. Su mejor configuración fue snake con poda activada (prune_on) y depth 3, con Win% 3.3 y max tile promedio 759.5. La ventaja es que corre mucho más rápido que Expectimax depth 3.

Y Random sirve como referencia: casi no logra progreso (max tile promedio 112.0) y tarda nada, lo cual deja claro que los resultados buenos vienen de la búsqueda y de las heurísticas.

[Visualización de Win% por configuración](#)

[Visualización de Max_avg por configuración](#)

[Visualización de Trade-off rendimiento-tiempo](#)

Comparación de profundidad en Expectimax

En Expectimax, subir de depth 2 a depth 3 en general implica un salto grande en tiempo. Por ejemplo, con smooth_heavy, el tiempo pasa de 37.2s a 122.3s, o sea, se vuelve bastante más caro computacionalmente. No hubo resultados que permitan afirmar que una profundidad es mejor que otra, pero hay casos como con snake donde el cambio es grande, max_tile promedio sube de 371.2 (depth 2) a 968.5 (depth 3), pero el tiempo también se dispara de 11.8s a 112.9s. Para otros presets como smooth_heavy y corner_heavy, no se ve una mejora clara en max_tile al subir a depth 3 (y igual se paga el tiempo extra).

Poda Alpha-Beta en Minimax

En Minimax, la poda alpha-beta ayuda a reducir el tiempo y en nuestras pruebas parece hasta mejorar un poco el rendimiento aunque el número de episodios sin poda fue limitado a 10 episodios en cada caso, lo que hace que la comparación no sea tan justa.. Minimax muestra una reducción clara de tiempo en la versión con poda de snake y de smooth_heavy, 17.8s vs 45.2s para snake y 21.5s vs 42.2s. Y aunque no es muy significativo el max_tile es levemente mejor en ambos modelos con la poda activada.

Conclusión

En 2048, el “oponente” real no es inteligente: el juego agrega un 2/4 al azar en una celda libre. Por eso, desde la teoría, lo esperable es que Expectimax sea el enfoque más adecuado: en lugar de asumir un rival que intenta arruinarnos la partida, modela correctamente la incertidumbre como un nodo CHANCE y toma decisiones optimizando el valor esperado. Eso es consistente con lo que vimos en los resultados: las mejores configuraciones fueron de Expectimax, alcanzando Win% = 16.7 y max_tile promedio cercano a 998.4 (smooth_heavy, depth 2) y 968.5 (snake, depth 3).

En cambio, Minimax asume un mundo “peor caso” (como si el spawn fuera un adversario perfecto que siempre elige la peor celda/valor para nosotros). Esa suposición es demasiado pesimista para 2048 y tiende a generar jugadas más conservadoras, lo que suele traducirse en menor desempeño promedio. En nuestras corridas, Minimax quedó por debajo en tasa de victoria (mejor caso Win% = 3.3 con snake + poda, depth 3), aunque con tiempos claramente menores que Expectimax con profundidad alta.

Por último, los resultados confirman que la heurística/preset con variación de pesos define qué “estructura” de tablero prioriza el agente (por ejemplo, mantener orden/monotonía tipo snake vs. suavidad/espacios). Por eso hay presets que dominan en una profundidad y no en otra. Y en Minimax, la poda alpha-beta funciona como debería desde la teoría: no cambia la

decisión óptima del modelo, pero reduce trabajo innecesario, lo que se refleja en menos tiempo.

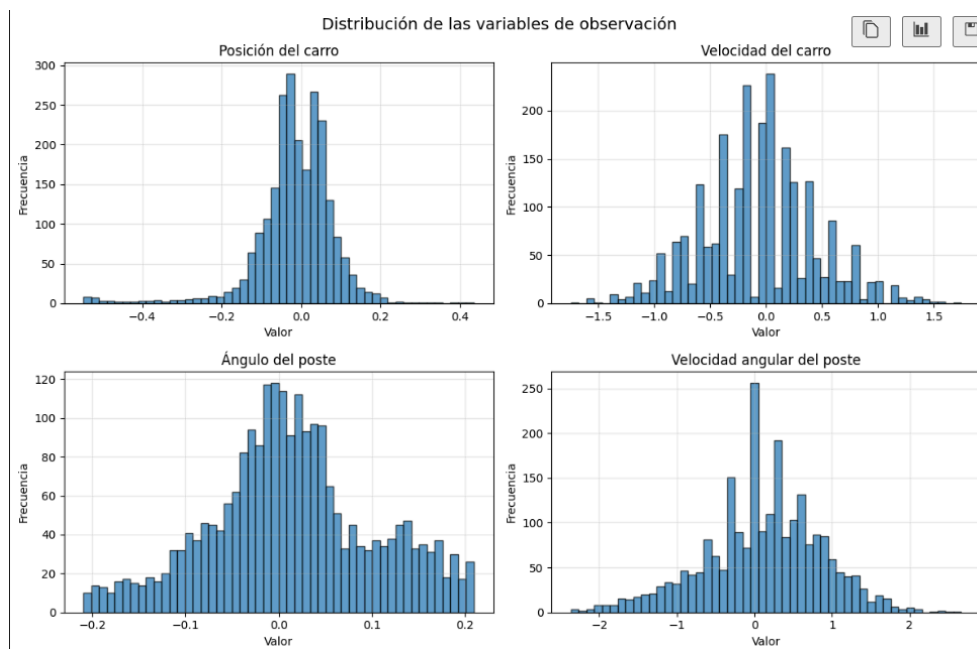
Anexo

Anexo - Módulo de Balance Dinámico (Gráficas y Visualizaciones)

En este anexo decidimos incluir todas las gráficas y tablas del notebook que creemos que aportan valor a la documentación, con el objetivo de respaldar visualmente el proceso y los resultados.

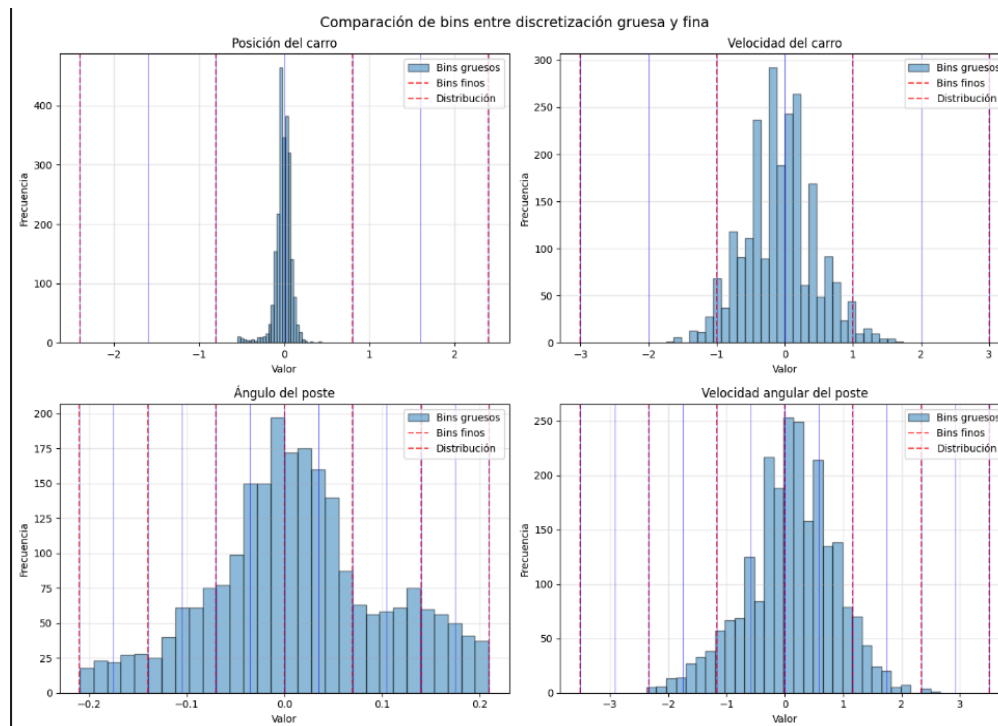
Las figuras se presentan respetando el orden del proceso en el notebook.

Figura 1 - Distribución de las variables de observación



En esta visualización se muestran los histogramas de las 4 variables de observación del entorno. Esto nos permitió desde un principio, entender los rangos y la dispersión real de cada variable durante episodios aleatorios, y usar esa evidencia para definir los rangos de discretización y la cantidad de bins de cada dimensión.

Figura 2- Comparación de bins entre discretización gruesa y fina

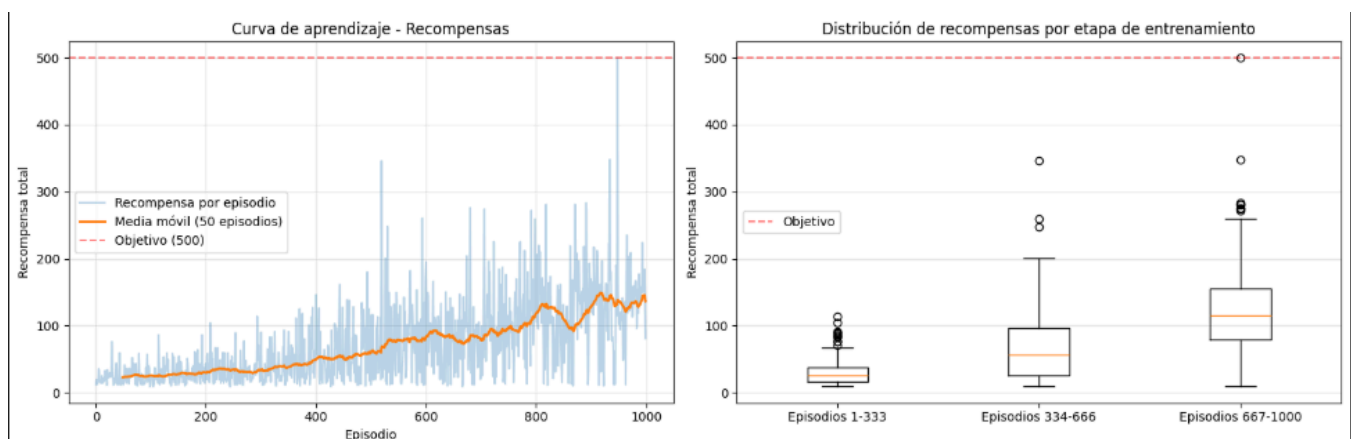


Esta segunda visualización nos permitió ver de manera clara los bins que definimos para cada variable en ambas discretizaciones, distinguiendo los bins de la discretización gruesa con una línea azul, y los bins de la discretización fina con una línea violeta punteada

Podemos notar que la discretización fina divide cada rango en más intervalos (mayor granularidad), mientras que la gruesa en menos.

Esto nos ayudó a entender cómo cambia la resolución del espacio de estados: la fina captura diferencias más sutiles (especialmente en variables críticas como ángulo y velocidad angular), a costa de incrementar significativamente la cantidad total de estados y el tiempo de entrenamiento.

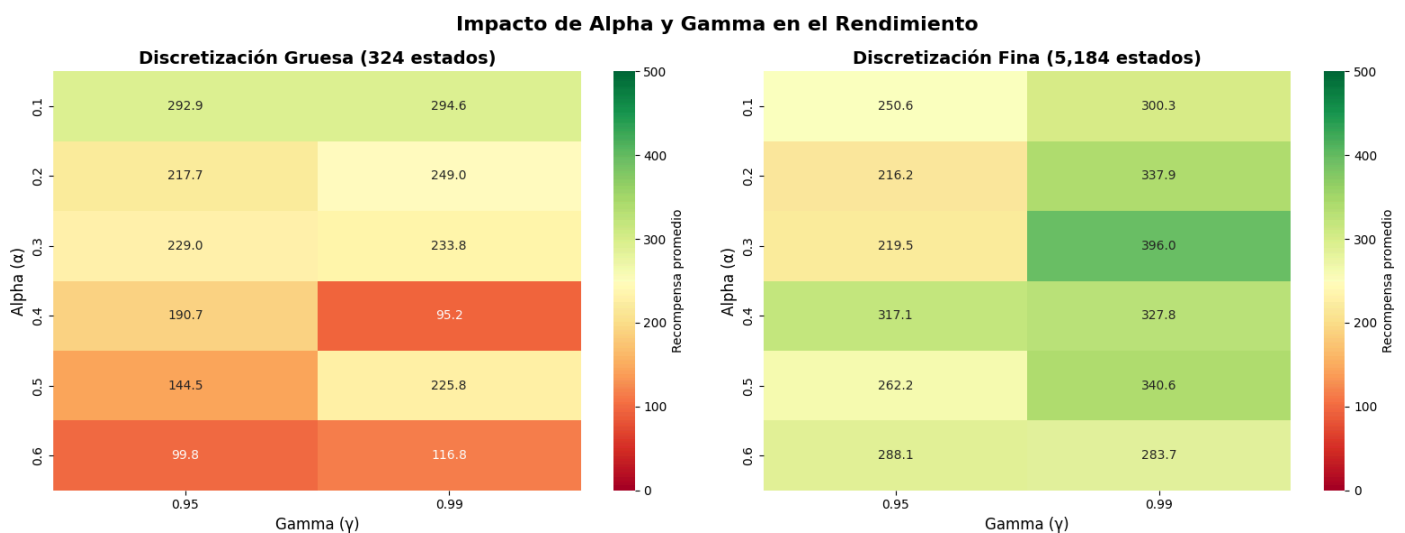
Figura 3 – Entrenamiento inicial de Q-Learning



Después de haber implementado Q-Learning, hicimos un entrenamiento corto de 1000 episodios para validar que el pipeline funcionara correctamente y obtener una primera señal de aprendizaje con una configuración base ($\alpha=0.1$, $\gamma=0.99$, $\epsilon_{\text{start}}=1.0$, $\epsilon_{\text{end}}=0.01$).

- **Curva de aprendizaje:** se muestran las recompensas por episodio y un promedio móvil. Se observa una tendencia creciente, indicando que el agente comienza a aprender una política mejor que la aleatoria, aunque todavía con alta variabilidad típica de etapas tempranas.
- **Boxplots por tercios:** la distribución de recompensas se desplaza hacia valores más altos a medida que avanza el entrenamiento, lo que confirma mejora progresiva y sirve como referencia para justificar la posterior exploración sistemática de hiperparámetros mediante grid search.

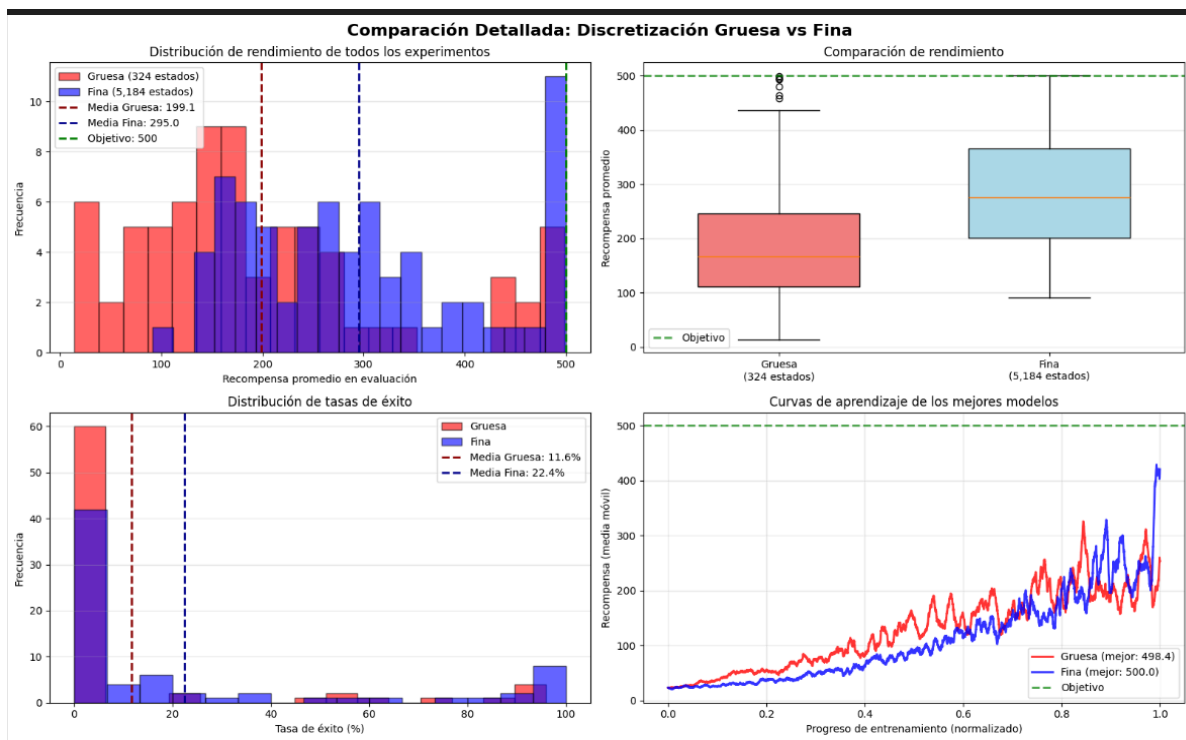
Figura 4 – Impacto de α y γ en el rendimiento (Q-Learning, discretización gruesa vs fina)



Luego de ejecutar el grid search de Q-Learning para ambas discretizaciones, se muestran heatmaps de recompensa promedio por combinación de α y γ . El objetivo de esto fue poder identificar rápidamente zonas de buen rendimiento.

- **Discretización gruesa (324 estados):** se observa que los mejores resultados se concentran en α bajos (0.1), mientras que valores de α más altos tienden a degradar el rendimiento, lo que sugiere que actualizaciones muy agresivas generan inestabilidad en un espacio de estados reducido.
- **Discretización fina (5.184 estados):** se aprecia un efecto marcado de $\gamma=0.99$ (mejor rendimiento general que $\gamma=0.95$) y una zona de buen desempeño con α moderados (0.3–0.5), consistente con que un espacio de estados más grande puede beneficiarse de un aprendizaje un poco más rápido.

Figura 5 - Comparación detallada: discretización gruesa vs fina (Q-Learning)



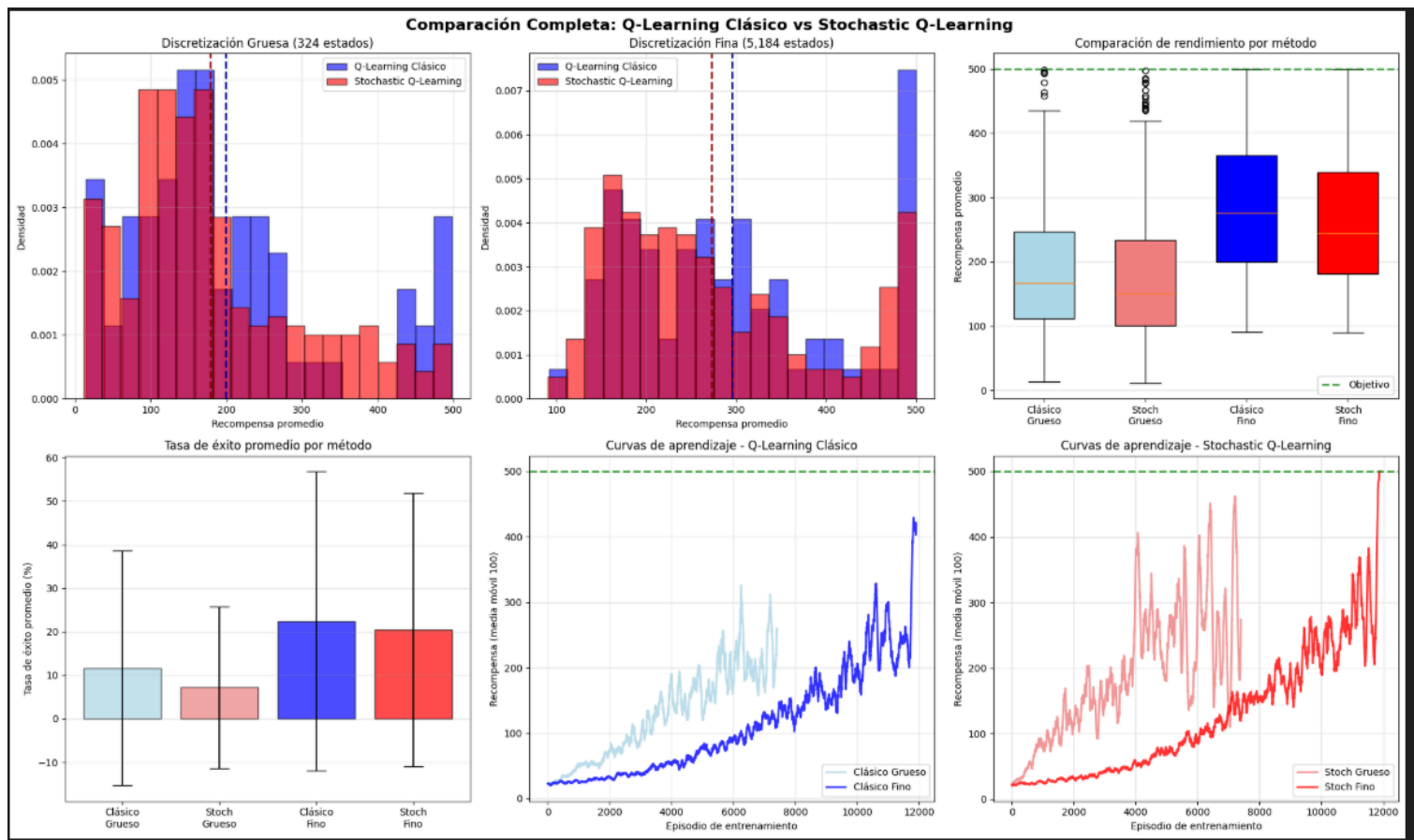
Esta figura resume, para todos los experimentos del grid search de Q-Learning, cómo cambia el rendimiento al pasar de 324 estados a 5184 estados.

- **Arriba izquierda (distribución de recompensas):** la discretización fina concentra más configuraciones en rangos altos y muestra una cola más marcada hacia valores cercanos al objetivo (500), mientras que la gruesa queda más dispersa y con mayor masa en recompensas bajas/medias.
- **Arriba derecha (boxplot de rendimiento):** la mediana y los cuartiles de la discretización fina están por encima de la gruesa, indicando un rendimiento típico superior para una combinación aleatoria de hiperparámetros.
- **Abajo izquierda (distribución de tasa de éxito):** la discretización fina presenta más casos con tasas de éxito altas, mientras que en la gruesa predominan configuraciones con éxito bajo o nulo, reflejando que alcanzar 500 steps es menos probable sin una granularidad suficiente.
- **Abajo derecha (curvas de aprendizaje de los mejores modelos):** ambos modelos mejoran con el entrenamiento, pero la discretización fina es la que logra llegar (o quedar consistentemente cerca) del objetivo de 500 steps, mientras que la gruesa típicamente se estabiliza un poco por debajo.

En conjunto, esta comparación muestra el trade-off central: más estados equivale a mayor capacidad para aprender una política óptima, a costa de más tiempo y costo de entrenamiento.

No está de más aclarar que ambas discretizaciones lograron obtener buenos resultados, e incluso como se vio en la documentación, la discretización ultra gruesa también fue capaz de alcanzar los 500 steps.

Figura 6 - Comparación Q-Learning vs Stochastic Q-Learning (gruesa vs fina)



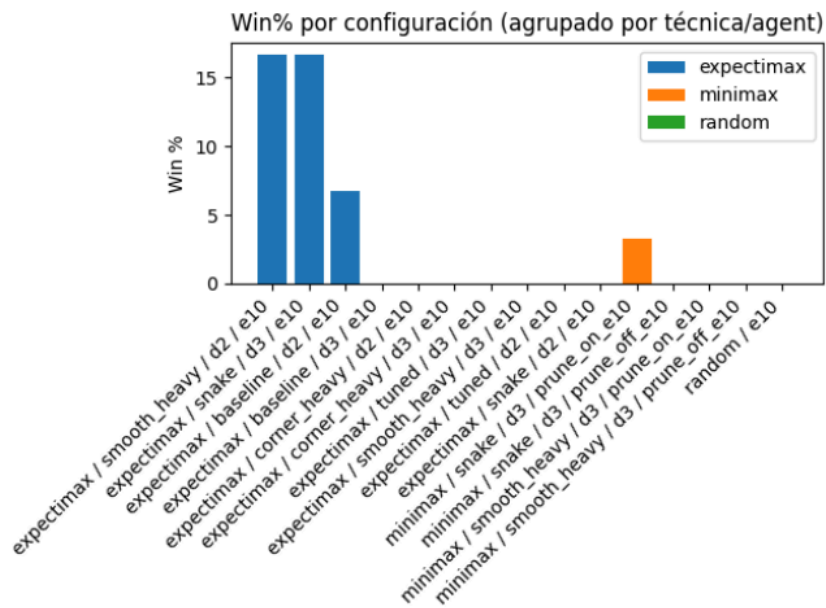
Finalmente, una vez que ejecutamos los dos grid search de Stochastic Q-Learning para ambas discretizaciones, decidimos visualizar los resultados para obtener mayor claridad de los resultados y las tendencias, y los comparamos con Q-Learning clásico.

La conclusión general que obtuvimos de estas visuales fueron que la discretización fina desplaza el rendimiento hacia valores más altos y aumenta la tasa de éxito en ambos métodos. En CartPole, donde solo hay 2 acciones, Q-Learning y Stochastic Q-Learning mostraron resultados similares; la diferencia más marcada proviene de la granularidad de la discretización. En las curvas de aprendizaje de debajo esto se ve claramente.

Anexo - Módulo de Estrategia Cognitiva (Gráficas y Visualizaciones)

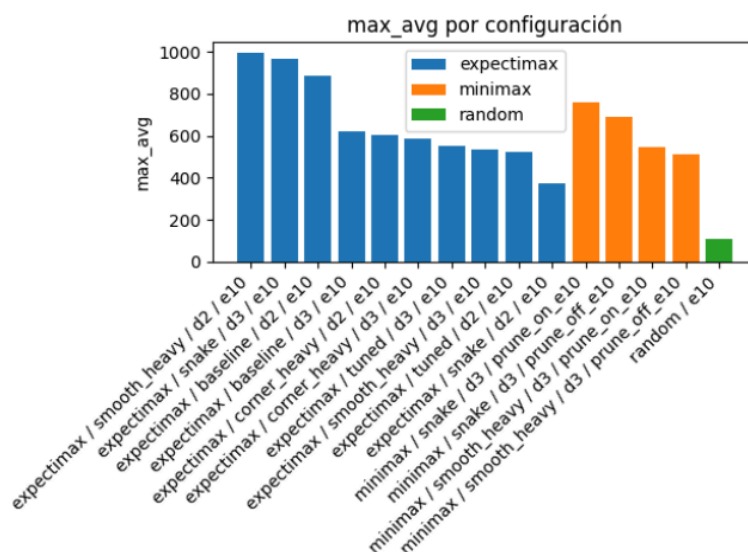
En este anexo reunimos las visualizaciones del Módulo de Estrategia Cognitiva que usamos para respaldar los resultados experimentales y comparar, de forma objetiva, las configuraciones de Expectimax y Minimax

Figura 1 (Win%): Tasa de victoria por configuración (Expectimax vs Minimax vs Random)



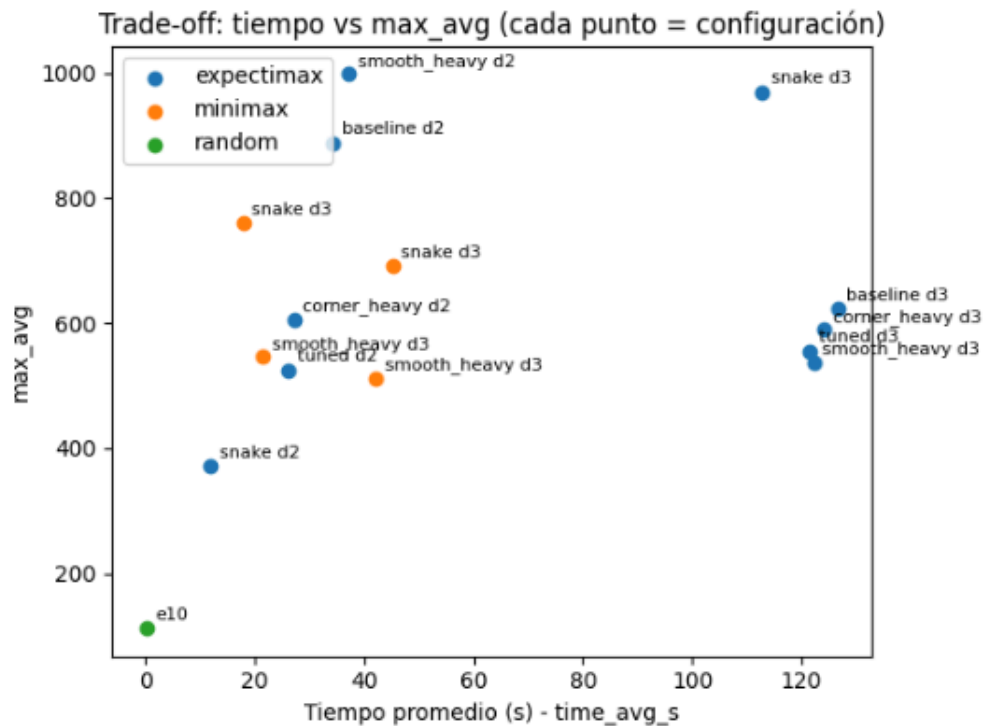
En este gráfico mostramos el porcentaje de partidas ganadas por cada configuración evaluada, agrupado por técnica.

Figura 2 - (max_avg): Máximo promedio alcanzado por configuración (comparación entre técnicas)



Acá presentamos el valor máximo promedio obtenido por cada configuración. Es útil para medir qué tan lejos llega cada enfoque en términos de desempeño, independientemente de si llega o no a ganar siempre.

Figura 3 - Trade-off rendimiento–tiempo: máximo promedio vs tiempo promedio por configuración



Este plot muestra el compromiso entre costo computacional y rendimiento: cada punto representa una configuración, posicionada por su tiempo promedio de ejecución y su máximo promedio alcanzado.